VALVER SMART DISPENSER

Documento de traspaso técnico para continuar con otra IA

ESTADO DEL PROYECTO

Snapshot del repositorio · 30 de octubre de 2026

Estado: prototipo temprano · firmware en modo seguro

Objetivo configurado: ESP32-S3 DevKitC-1 · movimiento y salidas inhibidos

Incluye inventario técnico, estado real del código, comandos de compilación, advertencias de hardware y una guía de continuidad para otra IA.

1. Resumen ejecutivo

VALVER SMART DISPENSER es un proyecto destinado a una máquina para dispensar adhesivos IMSE. El repositorio reúne una primera base para firmware de control, una estructura de interfaz de pantalla, documentación de arquitectura y hardware, un simulador sencillo de estados y carpetas que preparan trabajo futuro de pruebas, herramientas y recursos.

La parte ejecutable más concreta está en firmware/controller. PlatformIO está configurado para compilar con la definición de placa esp32-s3-devkitc-1, el paquete de plataforma espressif32 y el framework Arduino. La configuración no confirma que la placa física sea ese modelo. El firmware activo solo inicia el puerto serie, imprime diagnósticos y permanece en modo seguro. No mueve motores, no activa drivers, no lee sensores, no ejecuta dispensación y no dibuja una interfaz.

Una serie de encabezados define contratos iniciales para módulos de motor, sensores, seguridad, calibración, almacenamiento, comunicación y dispensación, pero no se encontraron archivos de implementación en los directorios de código correspondientes. La UI, los documentos detallados de hardware y las suites de pruebas también están pendientes.

Estado de una frase

Es una plantilla funcional para iniciar y diagnosticar el ESP32-S3 por serie, no una máquina dispensadora lista para fabricar, cablear o usar.

2. Qué está implementado, qué es una plantilla y qué no existe toda

Implementado en código

- Configuración PlatformIO del entorno del controlador.
- firmware/controller/src/main.cpp: setup(), loop() y reporte de diagnóstico por Serial.
- Constantes de proyecto y modo seguro en include/config/.
- Un enum de estados de sistema en include/system/SystemState.h.
- Declaraciones iniciales de interfaces de varios subsistemas.
- Un simulador Python independiente de una máquina de estados, con comprobaciones assert básicas.
- Un ejemplo mínimo de proyecto de pantalla, identificado por extensión .example.

Esqueleto o placeholder

- Los encabezados bajo firmware/controller/include/ describen funciones previstas, pero no constituyen su implementación.
- Las carpetas bajo firmware/controller/src/ para subsistemas están vacías en el inventario realizado, salvo el archivo raíz main.cpp.
- La configuración de pines tiene todos sus números en cero. Son valores provisionales, no conexiones GPIO.
- display/platformio.ini.example es un ejemplo de configuración, no la configuración activa del controlador.
- Las carpetas para pantallas, recursos, simuladores y pruebas están preparadas, pero no contienen funciones verificadas.
- Muchos archivos Markdown de docs/ tienen solo un encabezado y una nota de “pendiente”; no deben tomarse como requisitos aprobados.

No está implementado o validado

- Movimiento, homing, dosificación, control del TMC2209, lectura de sensores y calibración real.
- Interfaz gráfica funcional o comunicación real entre una pantalla y un controlador.
- Asignación GPIO validada, esquema, lista de materiales completa o procedimientos eléctricos aprobados.
- Protocolo de comunicación versionado y comprobado.
- Guardado/recuperación real de parámetros.
- Pruebas unitarias, integración y hardware completas.
- Confirmación de que la placa conectada físicamente sea la placa seleccionada en PlatformIO.

3. Mapa completo del repositorio

```
README.md
  Brevísima presentación del producto.

config/
  defaults/ y examples/ — directorios vacíos preparados para configuraciones.

display/
  platformio.ini.example, src/main.cpp.example
  include/ScreenManager.h, UIConfig.h, UIProtocol.h
  screens/Calibration, Diagnostics, Dispensing, HomeScreen,
          ManualControl, Recipes, Settings
  assets/fonts, icons, images; themes/
  Proyecto de interfaz en fase de estructura, no listo para cargar.

docs/
  CHANGELOG.md
  arquitectura/{arquitectura_general,comunicacion_controladores,
                diagrama_bloques,git_workflow}.md
  firmware/{configuracion_motores,estructura_firmware,
            protocolo_comunicacion,registro_parametros}.md
  hardware/{alimentacion_y_protecciones,conexiones_sensores,
            conexiones_tmc2209,inventario_componentes,pantalla_elecrow,
            pinout_esp32}.md
  pruebas/{plan_validacion,pruebas_dispensacion,pruebas_electronicas,
           pruebas_motores}.md
  seguridad/{analisis_riesgos,estados_seguridad,procedimiento_pruebas}.md
  PROJECT_HANDOFF_AI.md y PROJECT_HANDOFF_AI.pdf

firmware/controller/
  platformio.ini
  include/calibration/CalibrationManager.h
  include/communication/{Protocol.h,SerialManager.h}
  include/config/{HardwareConfig.h,PinConfig.h,SystemConfig.h}
  include/dispensing/{DispenserChannel.h,DispensingManager.h}
  include/motors/{MotorController.h,TMC2209Manager.h}
  include/safety/{FaultManager.h,SafetyManager.h}
  include/sensors/{EndstopManager.h,NeedleSensor.h}
  include/storage/SettingsManager.h
  include/system/{SystemManager.h,SystemState.h}
  src/main.cpp y carpetas src/{calibration,communication,dispensing,
     motors,safety,sensors,storage,system} vacías
  lib/ y test/ vacías.

hardware/
  datasheets/, mechanical/, schematics/, wiring/ — directorios vacíos.
```

```
scripts/
  build/, flash/, validation/ — directorios de trabajo preparados.

shared/
  constants/, models/, protocols/ — directorios vacíos.

simulation/
  README.md, state_machine_simulator.py
  motor_simulator/, protocol_simulator/, sensor_simulator/ vacías.

tests/
  hardware/, integration/, unit/ — directorios vacíos.

tools/
  calibration_tools/, diagnostics/, serial_monitor/ — directorios vacíos.
```

La presencia de un directorio indica organización prevista, no una función terminada. Los archivos que sí existen están enumerados a lo largo de esta guía y en las carpetas indicadas.

4. Arquitectura: visión prevista frente al comportamiento actual

Bloques previstos en la documentación

El diagrama inicial enumera cuatro elementos: ESP32-S3, driver TMC2209, sensores y pantalla. Es una arquitectura de alto nivel, sin interfaces eléctricas ni de software confirmadas. La documentación menciona comunicación entre controladores, pero no especifica si habrá dos microcontroladores ni un protocolo funcional.

Separación lógica sugerida por el código

Los encabezados proponen una arquitectura modular con estas responsabilidades:

1. SystemManager mantiene el estado general y decide si es seguro mover.
2. SafetyManager y FaultManager detectan/registran fallos y requieren reconocimiento.
3. EndstopManager y NeedleSensor representan señales físicas que deben validarse.
4. TMC2209Manager configura/consulta drivers; MotorController debería aislar control de motores.
5. CalibrationManager, SettingsManager, DispenserChannel y DispensingManager representarían calibración, parámetros y ejecución de la operación.
6. Protocol y SerialManager serían la frontera de comunicaciones.
7. display/ sería el futuro lado de interfaz humana.

Esta separación se deduce de los nombres y métodos declarados; no existe aún un flujo integrado. Por ejemplo, no se puede asumir que SafetyManager realmente impida una llamada al motor: todavía no hay una implementación que lo demuestre.

Flujo del programa que sí existe

```
Alimentar la placa por USB
        ?
Arduino llama setup()
        ?
Serial.begin(115200) y pausa de 500 ms
        ?
Mensajes de inicio y diagnóstico en el monitor serie
        ?
loop() espera y vuelve a reportar cada 5 segundos
        ?
Salidas de actuadores permanecen inhibidas
```

El flujo futuro de homing, calibración, armado y dispensación debe diseñarse, revisarse y probarse antes de implementarse. El enum por sí solo no realiza transiciones ni impone condiciones de seguridad.

5. Controlador: configuración exacta y funcionamiento

Archivo firmware/controller/platformio.ini

- [platformio] fija default_envs = esp32-s3-devkitc-1.
- [env:esp32-s3-devkitc-1] define el entorno de compilación.
- platform = espressif32 solicita el soporte de Espressif para PlatformIO.
- board = esp32-s3-devkitc-1 fija la definición de placa del entorno.
- framework = arduino selecciona Arduino.
- monitor_speed = 115200 configura el monitor serie.

No se declaran bibliotecas externas, opciones de carga específicas, puertos fijos ni flags de seguridad adicionales en este archivo.

Archivo firmware/controller/src/main.cpp

setup()

- Inicia el puerto serie mediante Serial.begin(valver::kDefaultBaudRate).
- Espera 500 milisegundos para permitir que el monitor serie se conecte.
- Imprime separadores, nombre del proyecto, versión 0.1.0-dev, el texto de modo seguro y el estado inicial esperado.
- Invoca printDiagnosticStatus().

loop()

- Conserva una marca temporal lastReport.
- Cada 5000 ms vuelve a imprimir el diagnóstico.
- Incluye una espera de 100 ms por iteración.
- No llama a motores ni sensores y no recibe comandos.

printDiagnosticStatus()

Imprime modo seguro, nombre/versión, indicador de movimiento habilitado, disponibilidad de motores, modo de simulación e inhibición de salidas. Los valores visibles proceden de constantes compiladas, no de una detección del hardware.

Mensajes que se esperan en Monitor serie: VALVER SMART DISPENSER, Firmware boot in safe mode, System initialized in safe mode y líneas [DIAG] periódicas. No se ha confirmado todavía que se hayan visto en una placa real.

6. Configuración de firmware, pinout y estado

include/config/SystemConfig.h

- kProjectName: nombre visible del producto.
- kFirmwareVersion: cadena 0.1.0-dev.
- kMovementDisabled: true, mantiene movimiento deshabilitado.

include/config/HardwareConfig.h

- kDefaultBaudRate: 115200U.
- kMotorsAvailable: false.
- kSimulationMode: false.

Estas constantes son flags compilados; no constituyen un gestor de configuración ni prueban la presencia/ausencia física del hardware.

include/config/PinConfig.h

PinConfig contiene siete campos de 8 bits: stepChannel1, dirChannel1, stepChannel2, dirChannel2, endstopA, endstopB y needleSensor. Todos se inicializan en 0. No cambiar estos valores hasta identificar la placa y diseñar el pinout conforme al datasheet. No conectar nada a GPIO 0 basándose en estos placeholders.

include/system/SystemState.h

Los estados declarados son BOOT, INITIALIZING, IDLE, NOT_HOMED, HOMING, READY, DISPENSING, PAUSED, FAULT y DISABLED_STATE. Los nombres sugieren fases/condiciones de una máquina de estados; aún no hay transiciones ni validaciones que los impongan.

Colisión con macro corregida

El primer intento de compilación mostró que el core ESP32 Arduino define DISABLED como macro numérica. El enum tenía también un miembro llamado DISABLED, causando error del preprocesador. El miembro se renombró a DISABLED_STATE para evitar la colisión. La corrección está aplicada, pero el siguiente pio run debe confirmar que el build completo termina con SUCCESS. No se cambió una función de seguridad: fue un ajuste de nombre en el enum.

7. Interfaces declaradas en include/

Esta sección enumera exactamente el propósito visible en las firmas. Las funciones no tienen una implementación demostrada y algunas clases carecen aún de datos o métodos suficientes para un sistema real.

Sistema

- system/SystemManager.h: constructor; initialize(); getState() const; setState(SystemState); isSafeToMove() const; campo privado currentState_. Debe definirse cómo se validan transiciones y qué condiciones hacen seguro el movimiento.
- system/SystemState.h: enum de estados descrito arriba.

Motores y drivers

- motors/MotorController.h: constructor; isAvailable() const; disableOutputs(). No define señales STEP/DIR, límites, velocidad, aceleración, pasos ni manejo de interrupciones.
- motors/TMC2209Manager.h: constructor; isDriverReady() const; disableDrivers(). No define interfaz UART, dirección, corriente, microsteps ni límites térmicos.

Sensores

- sensors/EndstopManager.h: constructor y isAnyEndstopActive() const. No identifica pines, polaridades, filtros, diagnóstico de cable abierto ni estados por eje.
- sensors/NeedleSensor.h: constructor y isNeedleValid() const. La nota de hardware cita EE-SX671 pendiente de validar, pero no se especifican su circuito o lectura.

Seguridad y fallos

- safety/SafetyManager.h: constructor; isFaultPresent() const; acknowledgeFault(). No hay reglas de fallo, paro, enclavamiento o recuperación implementadas.
- safety/FaultManager.h: constructor; reportFault(const char message). No define códigos, almacenamiento, severidades o canal de reporte.

Dispensación y calibración

- dispensing/DispenserChannel.h: constructor e isEnabled() const; representa conceptualmente un canal.
- dispensing/DispensingManager.h: constructor y canDispense() const; todavía no expone parámetros de receta ni control de volumen/tiempo.
- calibration/CalibrationManager.h: constructor y hasValidCalibration() const; no se especifican rutina, unidades, tolerancias ni persistencia.

Comunicación y almacenamiento

- communication/Protocol.h: estructura ProtocolMessage con dos enteros id y version. No es por sí sola un protocolo serial: faltan framing, longitud, checksum, codificación, límites, timeout y manejo de errores.
- communication/SerialManager.h: constructor y begin(unsigned long baudRate). No implementa en el encabezado lectura de mensajes ni comandos.
- storage/SettingsManager.h: constructor y loadDefaultSettings(). No define dónde se almacenan parámetros ni cómo migran/versionan.

8. Pantalla y experiencia de usuario

display/ es un proyecto aparte en preparación. Su README.md indica que el directorio prepara la base de UI.

- platformio.ini.example selecciona espressif32, esp32-s3-devkitc-1, Arduino y 115200. El sufijo .example importa: no es el platformio.ini activo del controlador.
- src/main.cpp.example solo inicia Serial, imprime Display project skeleton y espera en loop().
- include/UIConfig.h: nombre de UI y kDisplayUiEnabled = false.
- include/UIProtocol.h: estructura mínima UiCommand con commandId y channel.
- include/ScreenManager.h: constructor e initialize() declarados, sin implementación verificada.
- Las pantallas previstas son HomeScreen, ManualControl, Calibration, Diagnostics, Dispensing, Recipes y Settings. Sus carpetas están vacías.
- assets/fonts, assets/icons, assets/images y themes/ son directorios vacíos en el inventario.
- docs/hardware/pantalla_elecrow.md exige identificar el modelo exacto. No se conoce resolución, controlador gráfico, interfaz, tensión ni biblioteca recomendada.

No compilar ni cargar display/ como si estuviese terminado. Primero identificar hardware y definir si comparte el ESP32 o usa un controlador independiente.

9. Simulación y pruebas

Simulador Python

simulation/state_machine_simulator.py define:

- SystemState: estados creados con Enum y auto().
- SimulatedController: estado actual, flag motors_enabled y fault_active.
- initialize(): pasa por INITIALIZING, deshabilita motores, limpia flag de fallo y deja el estado en IDLE.
- command_move(authorized): rechaza si hay fallo o si no está autorizado; también rechaza si motores no están habilitados; de lo contrario cambia a DISPENSING.
- enable_motors(allowed): aplica el permiso solo si no existe un fallo.
- fault() y clear_fault(): activan el estado FAULT o vuelven a IDLE.
- Al ejecutarse como script, usa assert para probar arranque, rechazo de movimientos no permitidos, habilitación conceptual, bloqueo por fallo y recuperación.

Ejecución desde la raíz en Windows:

```
python .\simulation\state_machine_simulator.py
```

Salida esperada según el propio script: Simulation OK: state machine and command validation are working. El simulador es lógica de software de juguete. No acciona hardware, no simula tiempos reales ni valida el TMC2209, sensores, cableado o seguridad eléctrica. La opción de habilitar los motores en una prueba simulada no significa que se deba habilitar en el firmware.

Estado de pruebas del repositorio

tests/unit, tests/integration y tests/hardware existen pero están vacíos. firmware/controller/test también está vacío. Los documentos de pruebas son encabezados de planificación; no contienen resultados, mediciones ni criterios detallados de aprobación.

10. Cómo abrir, compilar y cargar el firmware

Abrir el proyecto correcto

En VS Code, abrir la carpeta firmware/controller como proyecto PlatformIO. Es la carpeta que contiene platformio.ini. Abrir solo el archivo .ini no siempre equivale a abrir la carpeta del proyecto.

Compilar

En PowerShell:

```
Set-Location "C:\Users\<usuario>\VALVER_SMART_DISPENSER\firmware\controller"
pio run
```

Cambiar <usuario> por la carpeta de usuario de Windows. La terminal debe quedar en ...\firmware\controller> antes de ejecutar pio run.

También se puede usar la paleta de comandos de VS Code: Ctrl+Shift+P ? PlatformIO: Build. Compilar no requiere placa ni puerto serie.

Errores ya observados:

- NotPlatformIOProjectError: se ejecutó pio run desde la raíz, donde no está el ini; entrar a firmware/controller.
- expected identifier ... DISABLED: el enumerador chocó con la macro Arduino; renombrado a DISABLED_STATE, queda volver a compilar.

Si la terminal dice que pio no se reconoce, usar PlatformIO: Build desde VS Code o abrir una terminal que tenga la CLI de PlatformIO disponible. No correr platformio project init: este repositorio ya tiene configuración en su subcarpeta.

Puerto, carga y monitor serie

El puerto se usa para la carga y el monitor, no para compilar. Cuando Windows enumere la placa, identificar el puerto COM real en Administrador de dispositivos o mediante:

```
pio device list
pio run -t upload --upload-port COM5
pio device monitor --port COM5 --baud 115200
```

COM5 es solo un ejemplo. Sustituir por el puerto real. No seleccionar un COM inventado. Monitor serie configurado a 115200.

Problema USB registrado en la sesión

Windows mostró “Dispositivo USB desconocido (Error de solicitud de descriptor de dispositivo)”. Eso significa que todavía no se había confirmado un COM; el problema ocurre antes de PlatformIO, no se arregla editando el enum ni el archivo de plataforma.

Secuencia de diagnóstico prudente:

1. Probar un cable USB que se sepa que transmite datos.
2. Conectar directamente a otro puerto de la PC, sin hub ni adaptador.
3. Si la placa tiene varios conectores, identificar su función por modelo/manual y probar el conector de programación.
4. Observar si la entrada aparece/desaparece en Administrador de dispositivos al reconectar.
5. Probar cable y placa en otra PC.
6. Identificar fabricante del puente USB/VID-PID antes de instalar el driver oficial correspondiente.
7. Usar BOOT/RESET solo tras identificar el modelo: el procedimiento depende de la placa.

No instalar controladores al azar ni cargar mientras Windows mantenga el error de descriptor. No está confirmada la causa física (cable, conector, placa o controlador), por lo que se debe diagnosticar, no adivinar.

11. Hardware nombrado, desconocido y seguridad

Elementos nombrados en los documentos

- ESP32-S3 como objetivo general/objetivo de PlatformIO.
- TMC2209 como driver de motor previsto.
- Finales de carrera y sensor EE-SX671 citado como pendiente de revisión.
- Pantalla Elecrow, sin modelo exacto.
- Fuente de 24 V mencionada en la advertencia de alimentación; no está documentado su diseño completo.
- Aplicación prevista para adhesivos IMSE.

No hay evidencia suficiente para fijar cantidad/modelo de motores, corriente, tensión de driver, modelo de la placa real, configuración de sensor o tipo de dispensado. Estos datos deben obtenerse de las etiquetas y hojas de datos del hardware físico.

Advertencias no negociables

- No conectar 24 V directamente a GPIO.
- No usar los 0 de PinConfig como indicación de cableado.
- Mantener kMovementDisabled = true y kMotorsAvailable = false mientras hardware/configuración no estén validados.
- No habilitar outputs ni cambiar el firmware a movimiento real sin análisis de riesgos, paro, límites, homing y prueba controlada.
- Separar alimentación de lógica y potencia según hojas de datos, diseñar protecciones/masa de forma explícita y revisar temperaturas/corriente antes de operación; los detalles concretos aún no están especificados.
- No asumir que una señal de sensor es compatible con 3.3 V del ESP32-S3 hasta comprobar datasheet y circuito.

12. Documentación existente: lectura correcta

Los documentos de arquitectura contienen títulos y declaraciones generales; la mayoría tienen una o dos líneas, no especificaciones completas.

- docs/arquitectura/arquitectura_general.md: dice “Sistema modular ...”.
- docs/arquitectura/diagrama_bloques.md: enumera ESP32-S3, TMC2209, sensores y pantalla.
- docs/arquitectura/comunicacion_controladores.md: protocolo pendiente de validar.
- docs/arquitectura/git_workflow.md: propone ramas main, develop, feature/, test/; no describe política completa.
- docs/firmware/estructura_firmware.md: sugiere estructura modular y segura.
- docs/firmware/protocolo_comunicacion.md: indica que el protocolo se documentará y validará.
- docs/firmware/configuracion_motores.md: dice que los drivers aún no se activan.
- docs/firmware/registro_parametros.md: pide requisitos de validación.
- docs/hardware/pinout_esp32.md: pinout pendiente de validación real.
- docs/hardware/alimentacion_y_protecciones.md: advierte no conectar 24 V a GPIO.
- docs/hardware/conexiones_tmc2209.md: conexiones se comprobarán con datasheet real.
- docs/hardware/conexiones_sensores.md: finales de carrera y EE-SX671 pendientes.
- docs/hardware/pantalla_elecrow.md: modelo exacto pendiente de identificación.
- docs/hardware/inventario_componentes.md: estado inicial de componentes, sin una lista validada.
- docs/seguridad/analisis_riesgos.md: recuerda considerar condiciones de fallo, sin matriz de riesgo.
- docs/seguridad/estados_seguridad.md: enumera BOOT, INITIALIZING, FAULT y puntos suspensivos.
- docs/seguridad/procedimiento_pruebas.md: propone pruebas seguras por etapas, sin procedimiento detallado.
- docs/pruebas/plan_validacion.md: menciona pruebas sin y con hardware.
- docs/pruebas/pruebas_electronicas.md, pruebas_motores.md, pruebas_dispensacion.md: pendientes.
- docs/CHANGELOG.md: marca la estructura base como cambio no liberado.
- README.md: identifica el proyecto como base para dispensación de adhesivos IMSE.

Todo lo anterior sirve como intención/contexto, no sustituye un plano, requisito aprobado o acta de validación.

13. Configuración del repositorio y convenciones

- .editorconfig: UTF-8, finales de línea LF, espacios, indentación de dos espacios, newline final.
- .gitignore: excluye .pio/, __pycache__/, .pyc, .log y .vscode/.
- docs/arquitectura/git_workflow.md: recomienda ramas main, develop, feature/ y test/, pero no documenta PRs, revisiones ni versionado.
- docs/CHANGELOG.md: una sección [Unreleased] con la estructura inicial.
- No hay manifiesto global de dependencias, CI, especificación versionada o instrucciones completas de contribución identificadas.

14. Decisiones que faltan antes de ampliar el desarrollo

1. ¿Cuál es la marca, modelo y revisión exactos de la placa y qué USB usa para programación?
2. ¿La pantalla tiene controlador propio o se conecta al controlador principal? ¿Qué modelo, interfaz y resolución?
3. ¿Cuántos motores/canales y qué actuador físico dispensa adhesivo?
4. ¿Qué modelos de TMC2209 y motores, corriente, UART y refrigeración se utilizarán?
5. ¿Cuáles son los modelos exactos de finales de carrera y sensor, salidas eléctricas, polaridad y tensión?
6. ¿Qué tensiones/fuentes, fusibles, paro de emergencia, límites eléctricos y conectores usa el sistema?
7. ¿Qué unidades, rango, precisión y tolerancias se exigen para la dispensación?
8. ¿Qué protocolo y topología de comunicación se adoptarán? ¿Serial TTL, USB u otro enlace?

9. ¿Qué criterios definen “homed”, calibración válida, fallo enclavado y recuperación segura?
10. ¿Cómo y dónde se almacenarán los parámetros, y cómo se actualizarán/versionarán?

Las respuestas deben salir del hardware real, requisitos del proceso, datasheets y revisión de seguridad. No rellenar huecos con suposiciones.

15. Plan de trabajo recomendado

Etapa A — restablecer una compilación reproducible

- Abrir firmware/controller como proyecto.
- Ejecutar pio run después del cambio a DISABLED_STATE.
- Resolver errores restantes sin cambiar la placa o quitar protecciones a ciegas.
- Registrar versión de PlatformIO, plataforma/framework instalados y resultado de build.

Etapa B — comunicación USB/serie sin potencia

- Identificar la placa física y su conector de programación.
- Resolver el error de descriptor USB hasta obtener COM/USB reconocible.
- Cargar firmware de diagnóstico sin periféricos de potencia.
- Confirmar salida serie a 115200 y registrar evidencias.

Etapa C — levantamiento de hardware y diseño de seguridad

- Fotografiar etiquetas y registrar part numbers.
- Obtener datasheets de placa, driver, motor, sensor, display y fuente.
- Crear inventario, pinout, esquema, matriz de tensiones y análisis de riesgos.
- Revisar niveles lógicos, protecciones, tierra y procedimiento de emergencia antes del cableado.

Etapa D — especificaciones y software de bajo riesgo

- Especificar máquina de estados y transiciones permitidas.
- Definir protocolo con framing/versionado/validación/timeout y errores.
- Definir formato, unidades, rangos y persistencia de parámetros.
- Ampliar simulador y agregar pruebas unitarias/integración antes de hardware.

Etapa E — integración secuencial de hardware

- Validar entradas/salidas sin actuadores y con alimentación limitada.
- Probar sensores individualmente y estados de desconexión/fallo.
- Validar configuración del driver según datasheet, primero sin carga y después con motor de forma controlada.
- Implementar homing con límites y recovery seguros.
- Integrar dispensación solo con criterios de calibración y aceptación cuantificables.

En cada etapa, mantener actuadores inhibidos hasta que la etapa anterior se haya verificado y documentado.

16. Guía corta para el desarrollador que recibe el proyecto

1. Lee esta guía y distingue código real de placeholders.
2. Abre firmware/controller, no la raíz, para el build de PlatformIO.
3. Ejecuta pio run; la última compilación conocida falló por el macro DISABLED y el cambio a DISABLED_STATE debe verificarse.
4. No dependas de la placa hasta confirmar que Windows ya no muestra error de descriptor y que existe un puerto.
5. Mantén movimiento apagado. No escribas GPIO ni recomendaciones de cableado hasta identificar el hardware físico.
6. Inspecciona archivos, compila y ejecuta la simulación antes de refactorizar módulos.
7. Cuando se implementen interfaces, añade sus fuentes en las carpetas src/ que corresponden y pruebas automatizadas.

8. Actualiza documentación y changelog con decisiones verificadas; marca expresamente lo no probado.

Prompt completo para iniciar una nueva conversación con otra IA

Continúa VALVER SMART DISPENSER basándote en el repositorio y en esta guía integral. Primero inspecciona el estado real de archivos y ejecuta/verifica la compilación desde firmware/controller; no supongas que el rename DISABLED_STATE ya fue validado. La placa física no está confirmada y Windows reportó “Dispositivo USB desconocido (Error de solicitud de descriptor de dispositivo)”, así que no afirmes que existe un COM. Mantén kMovementDisabled=true y kMotorsAvailable=false. No inventes GPIO, cableado, modelo de display, motor ni corriente. Explica claramente qué está implementado, qué es una interfaz vacía y qué dato real falta. Propón pasos pequeños y comprobables, priorizando USB/serie, inventario/datasheets y seguridad. Actualiza código, pruebas y documentación en conjunto; nunca habilites actuadores antes de revisar pinout, homing, sensores, fallos y procedimiento de paro.

17. Inventario de fuentes para navegar el proyecto

Firmware de control

firmware/controller/platformio.ini; src/main.cpp; encabezados include/config/{HardwareConfig,PinConfig,SystemConfig}.h, include/system/{SystemManager,SystemState}.h, include/motors/{MotorController,TMC2209Manager}.h, include/sensors/{EndstopManager,NeedleSensor}.h, include/safety/{FaultManager,SafetyManager}.h, include/dispensing/{DispenserChannel,DispensingManager}.h, include/calibration/CalibrationManager.h, include/communication/{Protocol,SerialManager}.h, include/storage/SettingsManager.h.

Interfaz

display/README.md; platformio.ini.example; src/main.cpp.example; include/{ScreenManager,UIConfig,UIProtocol}.h; las siete carpetas de pantallas previstas.

Pruebas y documentación

simulation/state_machine_simulator.py; simulation/README.md; carpetas tests/{unit,integration,hardware}; documentación de docs/{arquitectura,firmware,hardware,seguridad,pruebas} y docs/CHANGELOG.md.

Carpetas preparadas sin contenido de implementación localizado

config/{defaults,examples}, hardware/{datasheets,mechanical,schematics,wiring}, scripts/{build,flash,validation}, shared/{constants,models,protocols}, simulation/{motor_simulator,protocol_simulator,sensor_simulator}, tests/{hardware,integration,unit}, tools/{calibration_tools,diagnostics,serial_monitor}, display/assets/{fonts,icons,images} y display/themes.